

Lecture 2: Introduction to Python

Lecture 2 : Outline

- Introduction
- Python 3.1 Installation
- “ Hello World” – Example
- Input , Processing and Output
- Variables and Data Types
- Python Statements and Expressions
- Python Comments
- How Python Works?
- Lab#1
- Anaconda Navigator Installation

Introduction to Python

Introduction

- **Computational Thinking**
 - A thought process to formulate a problem and express its solution in terms a computer can apply effectively
- Let's assume to solve a problem :
 - We considered it at right level of abstraction
 - Decomposed it (if there was a possibility)
 - Designed an Algorithm (Defined the steps to solve it)
- Now , how to instruct the computer ?
 - How to make the computer follow the algorithm?
 - How to automate ?
- **Programming**
 - A process of creating instructions for a computer to follow
 - These instructions are written in a programming language

Programming Languages

- **Machine Language**

- A computer can only directly understand Machine Language
- Also called binary language – 1's and 0's Only
- Easier for computer to understand but difficult for programmers to program

- **Assembly Language**

- Uses special symbols called mnemonics instead of binary numbers
- Assembly language program cannot be executed directly by CPU
- Need a translator called “Assembler”

- **Challenges**

- Both are difficult for humans to program
- Each brand of CPU has its of instruction set (binary and assembly)
 - So, program written for one CPU brand may not work for the other (Compatibility Issue)

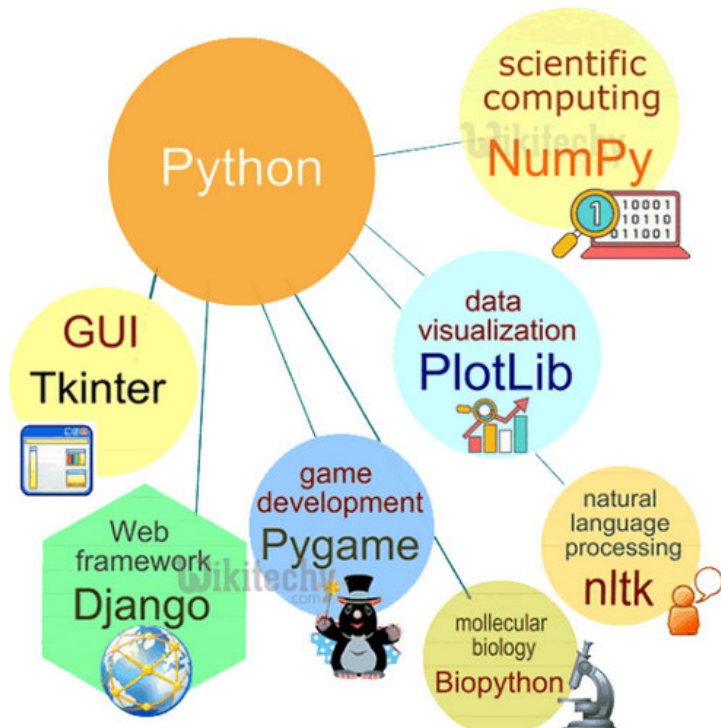
Programming Languages

- High Level Languages
 - Easier to program for humans or programmers
 - No need to know the instruction set (in binary)
 - Easy to write complex and large programs
 - A computer cannot understand directly
 - Translator programs (Compiler or Interpreter) are used to convert into binary
- Examples
 - Ada
 - BASIC ,FORTRAN, COBOL, Pascal
 - C, C++, C#
 - Java and Java Script
 - Python

What is Python?

- Python is an open-source , general-purpose , high-level programming language
- Developed by Guido van Rossum
- Released in 1991
- Mainly used for:
 - Backend Development (Server-side programming)
 - Web Development (Django and Flask Frameworks)
 - Data Analysis and Visualizations
 - Machine Learning Tasks
 - Deep Learning Task
 - GUI
 - Software Development
 - Research & Development
 - All other task in conventional high-level languages

Python – A huge set of Libraries for almost each specialized task



Pandas 
TensorFlow 
Keras 
PyTorch 
theano

plotly 
matplotlib 
scikit-learn 

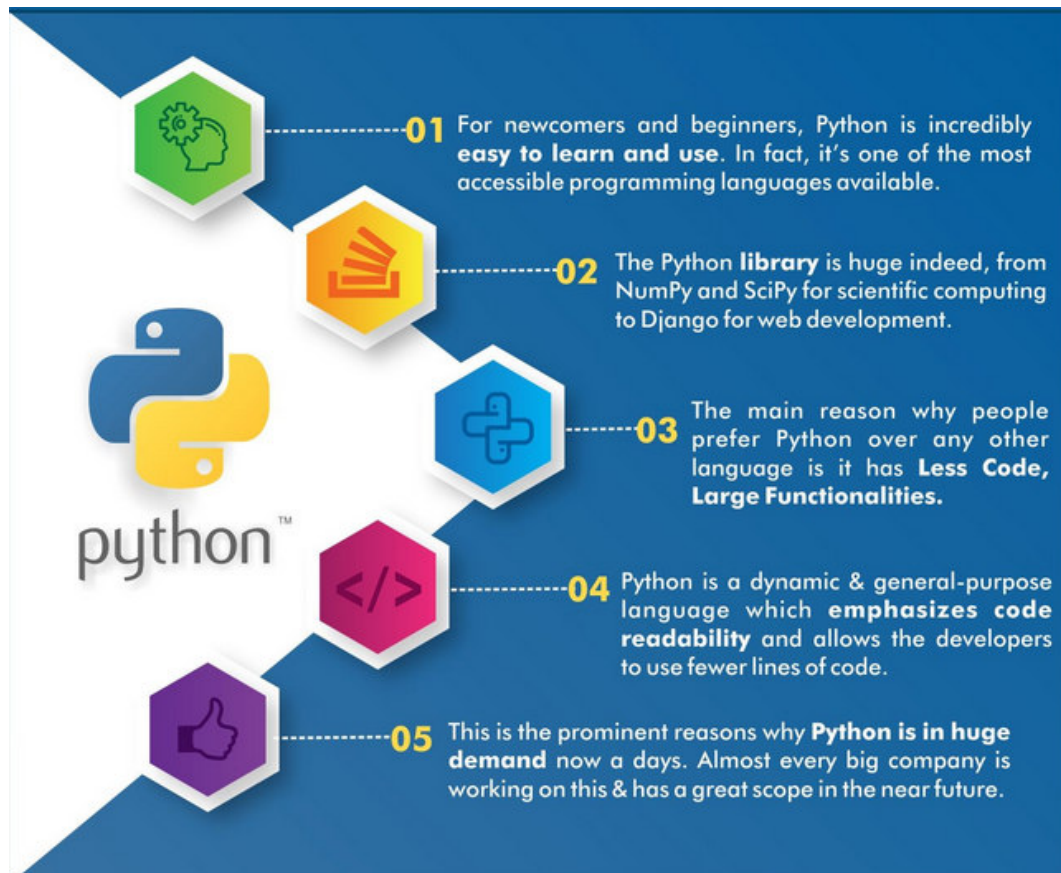
Source : <https://rotechnica.com/what-is-python-used-for/>

Giants are using Python!

- **Google:** The google is among one of the prominent companies which uses python in its development and design. Python is proven to be efficiently handling the traffic deployed on google and its connected apps and is very well known for computing purposes.
- **YouTube:** One of the most beloved apps these days which is very much in trend for leisure times is of course YouTube and to surprise is written in python.
- **Dropbox:** Starting from storing documents at first, Dropbox has spread its wings now to store literally everything and the functionality of sharing and synchronising the stuffs saved has made it even more lovable to its audience and all this is powered by python.
- **Instagram:** Instagram has become the most trending application for the purpose of sharing pictures and videos for its people. Apart from sharing, several other features are being provided by this application which is all powered by python and make it even more popular.
- **Quora:** The next big giant after google which is proved good in providing solutions to all the queries posted in a most realistic way is none other than Quora. And all this thing is summed up to an application with the help of python.

Source : <https://rotechnica.com/what-is-python-used-for/>

Reasons to choose Python ?



Open Source

Easy to Learn

Huge Library - Strong Online Community

Less Code , Large Functionalities

Code Readability

High Demand

Python Installation

- Check if python is already installed on your computer
 - **cmd** → **python --version**
- Visit www.python.org
- Download Python 3.12.7 / latest version for Windows
 - Choose the default path
 - Remember to check: ☒ Add Python to environment variables
- After setup completes
 - Again, run **python --version** on command prompt to verify successful installation

Starting out in Python – Hello World!

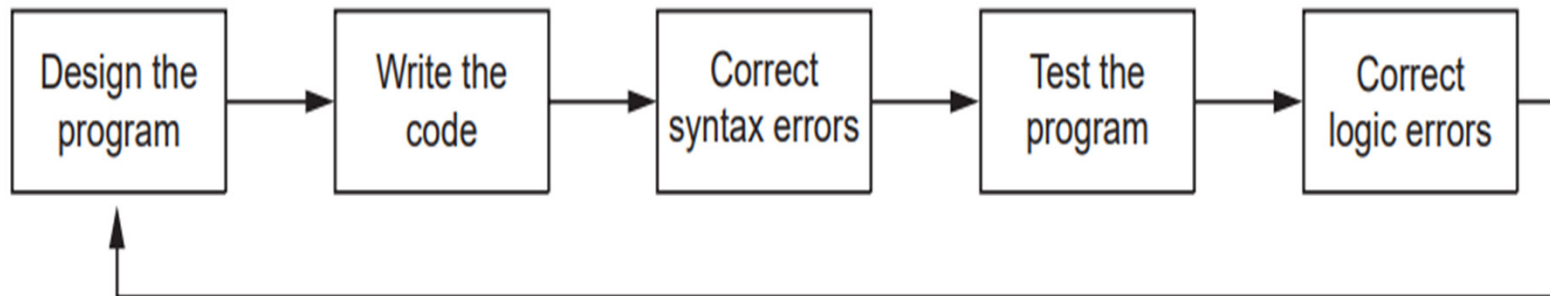
- There are various ways to run python code:
 - Interactive mode (Python shell)
 - Python IDLE
 - Text Editor (Visual Studio Code)
 - IDE (PyCharm)
 - Anaconda Navigator (
 - **Jupyter Notebook**
 - Jupyter Lab
 - Spyder
 - PyCharm
 - Visual Studio Code
 - We will be using Anaconda Navigator (Jupyter Notebook)
 - We can also use <https://colab.google/> (Google Colab) but it needs internet access .

Hello World !

- Interactive Mode
 - Open **Python shell** and print **Hello World!**
- Command Line
 - Create a text file
 - Print("Hello World! ")
 - Save it as hello.py in your user directory
 - Run this hello.py file from command line
- Python IDLE
 - Print " Hello World"
 - Create a new file
 - Create two variables : a=10 , b=20
 - Using if statement print " a is less than b " or " a is greater than b"
- Jupyter Notebook

The Program Development Cycle

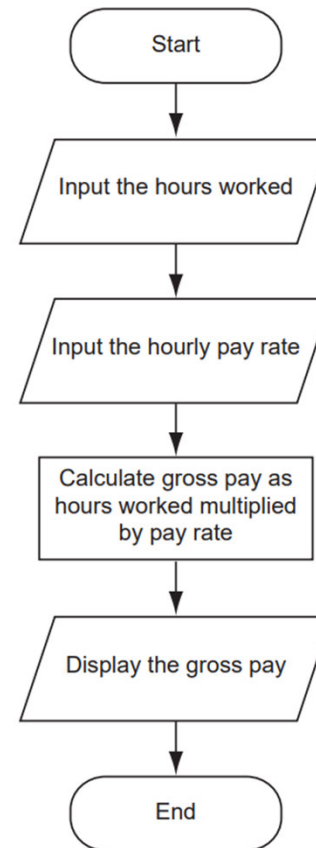
- The process of designing a program includes:
 - Understand the task that the program is to perform
 - Problem Formulation (Computational Thinking)
 - Determine the steps that must be taken to perform the task
 - Algorithm Design (Computational Thinking)
 - Pseudocode – The language (not a computer language) in which an algorithm is written



Two Important Tools – Solution Design

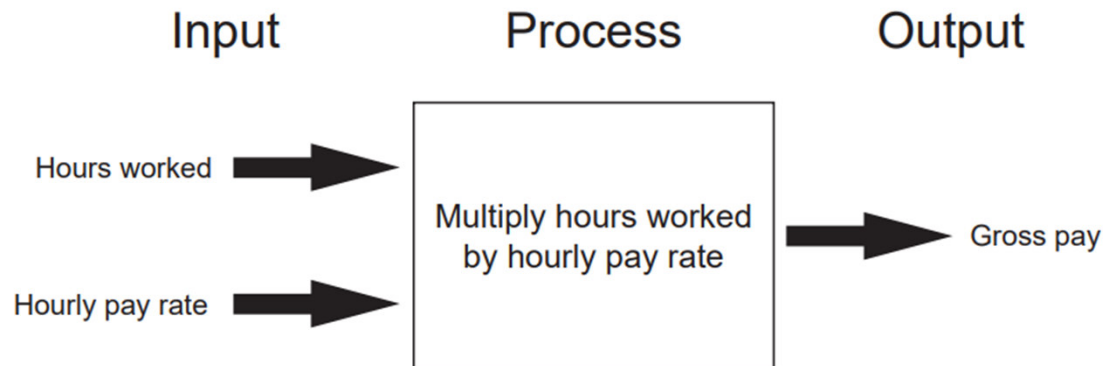
- Pseudocode and Flow Charts

Input the hours worked
Input the hourly pay rate
Calculate gross pay as hours worked multiplied by pay rate
Display the gross pay



Input → Processing → Output

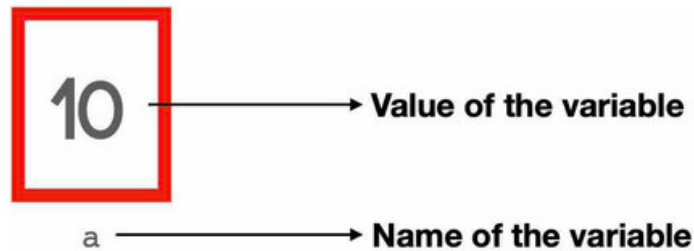
- All computer programs typically perform the following three operations:
 - Input (There are different ways to provide input)
 - Processing (Use mathematical functions)
 - Output (There are different ways to provide the output)



Variables and Data Types

Variables

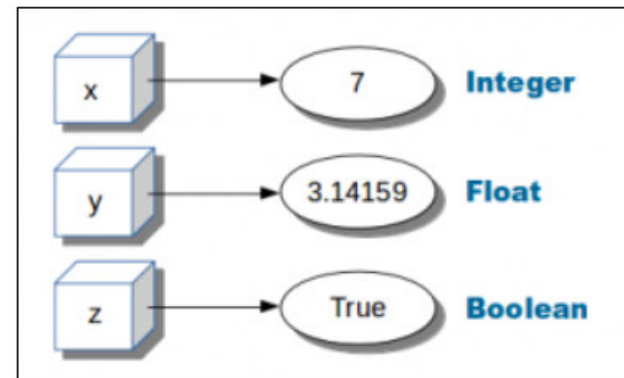
- A variable is a named memory location which is used to store data.
- Each variable has a type which depends on the values it stores
-



Consider the following statement.

```
a = 10
```

Here, **a** is a variable that stores a value of 10.



Variables Naming Conventions

- Each variable in python has a name
- Naming Conventions:
 - The first character must be one of the letters a through z, A through Z, or an underscore character (`_`)
 - After the first character you may use the letters a through z or A through Z, the digits 0 through 9, or underscores.
 - Uppercase and lowercase characters are distinct. This means the variable name `ItemsOrdered` is not the same as `itemsordered`
 - You cannot use any of Python's **key words** as a variable name.
 - A variable name cannot contain spaces
 - You can't use **special characters** like `!`, `@`, `#`, `$`, `%` etc. in variable name.

Python Keywords

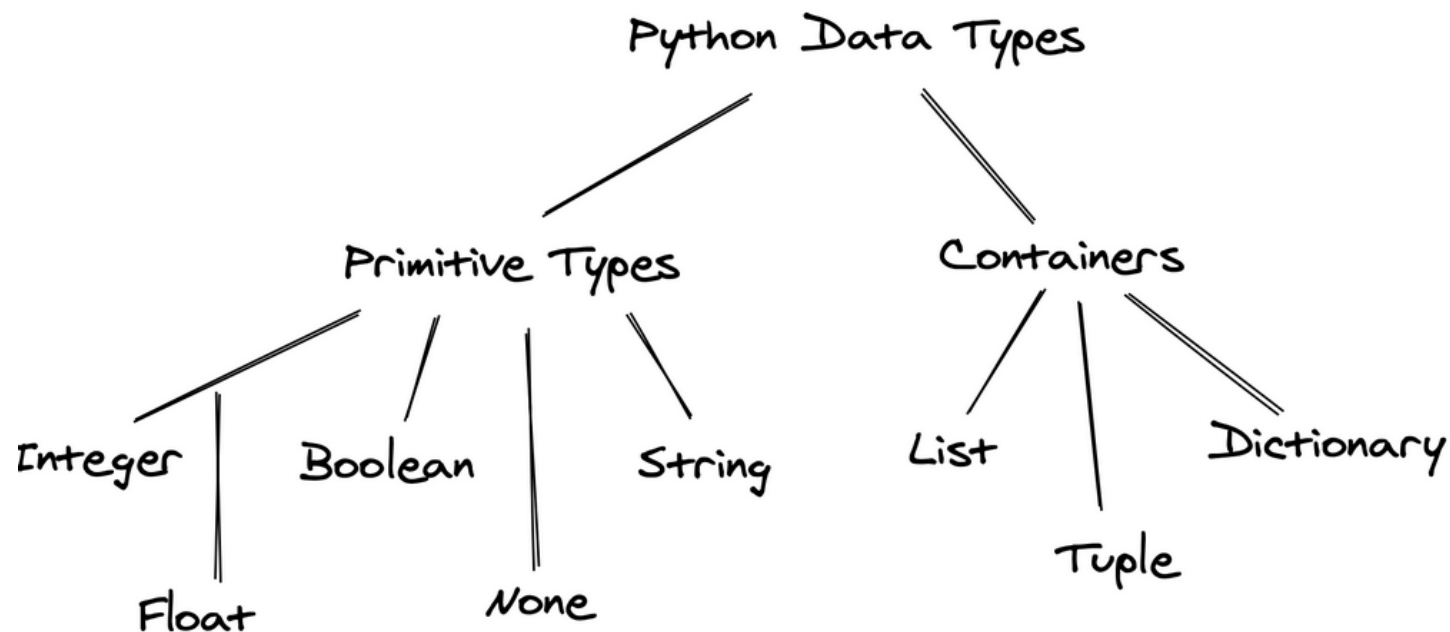
- Keywords have specific use in python language
- These keywords cannot be used as variable names

Keywords in Python				
False	class	<u>finally</u>	is	return
None	continue	for	lambda	try
True	def	from	nonlocal	while
and	del	global	not	with
as	<u>elif</u>	if	or	yield
assert	else	import	pass	
break	except	in	raise	

Python Data Types

- Data types represents the type of value (to be stored in a variable)
- For examples :
 - 10 is an **integer** value
 - 20.5 is a **floating-point** value
 - 10+5j is a **complex** number
 - “Hello World” is a **string** value
 - True and False are **Boolean** values
 - [“python” , “Java” , “C++”] is a **list** of values
 - There are other types as well

Python Data Types



Mutable and Immutable Data Types

- Mutable Data Types

- Those whose values can be changed in place:

1. Lists
2. Dictionaries
3. Sets

- Immutable Data Types

- Those that can never change their value in place.

1. Integers
2. Floating-Point numbers
3. Booleans
4. Strings
5. Tuples

Type Casting

- Type Casting
 - A method to convert the variable data type into another data type
- There can be two types of Type Casting in Python
 - Implicit Type Casting
 - Explicit Type Casting
- Examples :
 - `a=5, b=4.5 , c=a+b , d=a*b` (what are the data types of c and d)
 - `int(3.1416)` will give you 3
 - `float(3)` will give you 3.0. (a float value is stored differently than an integer)
 - `value=10 , str(value)` will give you “10” which cannot be treated as a number now
 - `x=3.5 , y=int(x)` will cause `y= ??` and `x=??`

Statements and Expressions

Python Statements

- A statement is an instruction that a Python interpreter can execute
- It is a single line of execution that can be executed independently of any other code
- For example :
 - print () statement
 - Assignment statement
 - Conditional statement
 - Loop statements
- Each python statement ends with a new line (\n – line feed character)
- A statement can be extended to multiple lines

```
Addition=10+20+ \  
30+40+\  
50+60+70
```

Python Expressions

- An expression in Python is a line of code that produces some value
- It is a combination of operands and operators
- Examples
 - $2+4$
 - $A+B$
 - $a>b$ (may be True of False)
 - $2+5*4$
 - $X=a+b$ ($a+b$ is expression ; the whole thing $X=a+b$ is a statement)

Difference Between Statement & Expressions

The main difference between an expression and a statement in Python is that an expression evaluates to a value, while a statement performs an action.

Expressions are combinations of operands and operators. They can be simple, such as the number 1, or more complex, such as $1 + 2 * 3$. When an expression is evaluated, it returns a value.

Statements are used to perform actions, such as assigning a value to a variable, printing a message to the console, or calling a function. Statements do not return a value.

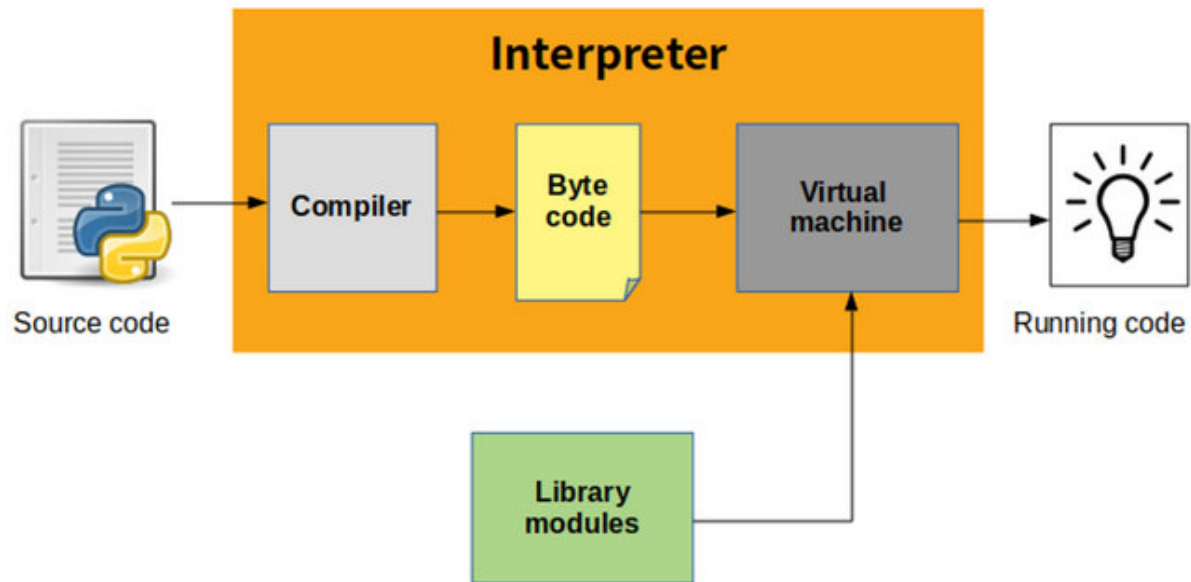
Comments in Python

- **Comments describe what is happening inside a program** so that a person looking at the source code does not have difficulty figuring it out.
- The Python Interpreter simply ignores the comments and does not check for Python Syntax
- Types
 - Single Line Comments
 - # this is a single line comment
 - Multiline comments
 - Either use # sign at the beginning of each new line
 - Or use triple quotes to include multiple lines

Lab#1

1. Create a new Jupyter Notebook and name it as Lab#1
2. Type and execute `print("Hello World")`
3. Create two variables `a=10`, `b=20` and `sum=a+b`
 - a) Write above three statements in separate lines (in a single cell) each preceded by a single line comment
 - b) Identify the statement and expressions in above
4. Create two more variables `choice1=True` and `choice2 = "True"`
 1. Check the type of both variables `choice1` and `choice2`
 2. Display the hexadecimal address of both `choice1` and `choice2`
5. Create a string variable and check its type
6. Create a float variable and cast it into an int type
7. Know the shortcuts:
 1. ESC+A to insert cell above the current cell
 2. ESC+B to insert cell below the current cell
8. Considering the variable naming rules , do the following:
 1. Create and initialize any 5 variables with valid names
 2. Try to create 3 variables which violate these rules.

How it Works ?



How it works ?

Anaconda Navigator Installation

What is Anaconda ?

- **Anaconda** is a distribution of the Python programming language and R for scientific computing including:
 - Data Science
 - Machine Learning Applications
 - Large-scale Data Processing
 - Predictive Analytics, etc.
- Aims to simplify package management and deployment.
- The distribution includes data-science packages suitable for Windows, Linux, and macOS.
- It is developed and maintained by Anaconda, Inc., which was founded by Peter Wang and Travis Oliphant in 2012

Anaconda Distribution

- Anaconda, Inc. product, it is also known as **Anaconda Distribution**
- **Different Products:**
 - **Anaconda Individual Edition (Free)**
 - **Anaconda Team Edition (Commercial)**
 - **Anaconda Enterprise Edition (Commercial)**
- **Anaconda distribution** comes with over 250 packages automatically installed, and over 7,500 additional open-source packages which can be installed based on need
- It also includes a GUI, **Anaconda Navigator**, as a graphical alternative to the command-line interface (CLI).

Anaconda Navigator

- Anaconda Navigator is a desktop graphical user interface (GUI) included in Anaconda distribution
- The following applications are available by default in Navigator:
 - JupyterLab
 - Jupyter Notebook
 - QtConsole
 - Spyder
 - Glue
 - Orange
 - RStudio
 - Visual Studio Code

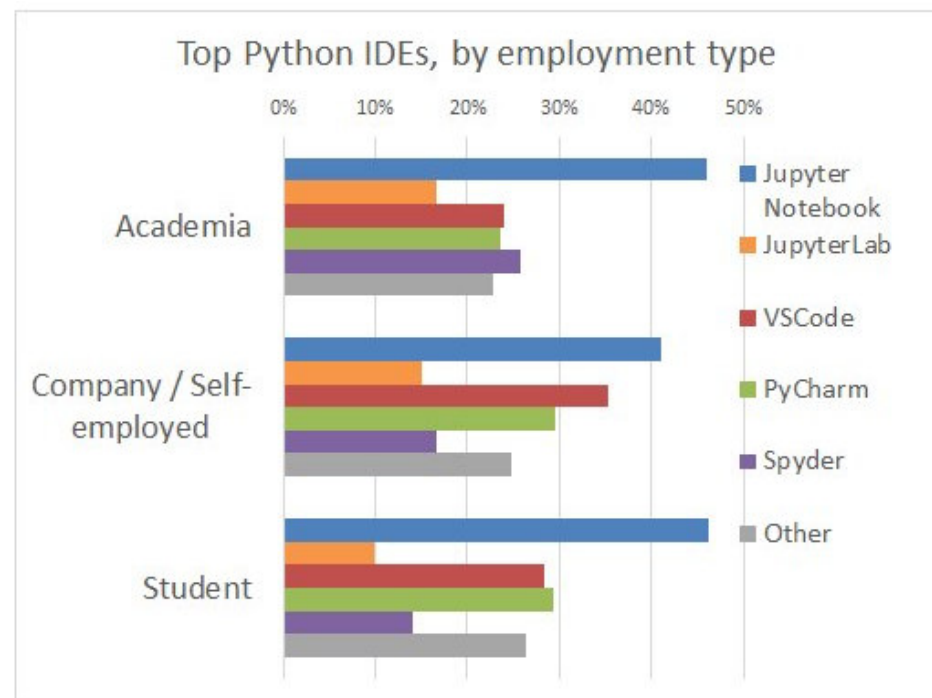
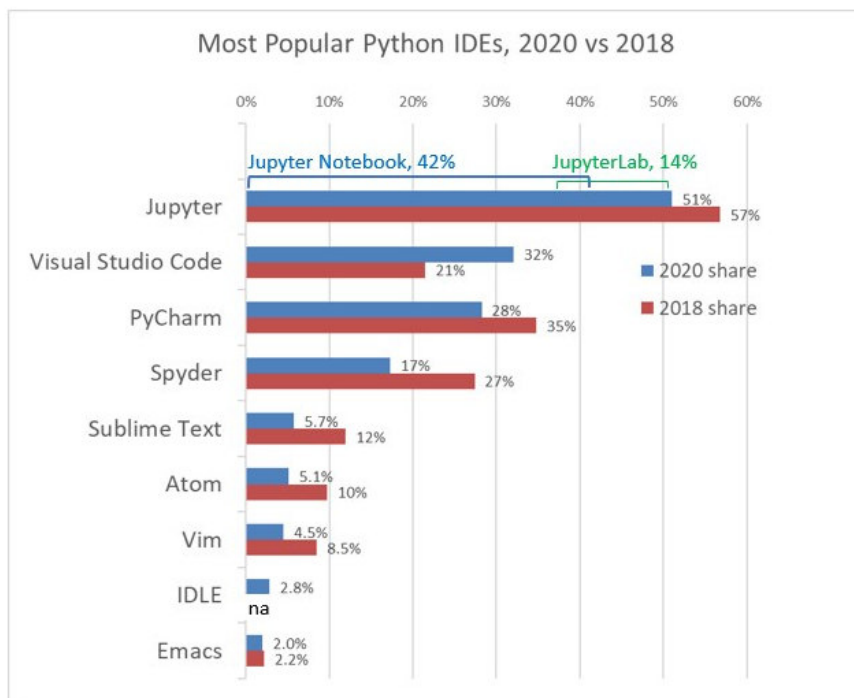
Anaconda Individual Edition - Installation

- <https://problemsolvingwithpython.com/01-Orientation/01.03-Installing-Anaconda-on-Windows/>
- https://www.tutorialspoint.com/sdlc/sdlc_quick_guide.htm

A Brief Description of Python IDEs / Editors

Jupyter Notebook	Jupyter Notebook (formerly IPython Notebook) is a web-based interactive computational environment for creating notebook documents. (Wikipedia)
Jupyter Lab	JupyterLab is the next generation of the Jupyter Notebook. It aims at fixing many usability issues of the Notebook, and it greatly expands its scope. JupyterLab offers a general framework for interactive computing and data science in the browser, using Python, Julia, R, or one of many other languages. (Source)
Spyder	Spyder is an open-source cross-platform integrated development environment (IDE) for scientific programming in the Python language . (Wikipedia)
PyCharm	PyCharm is a dedicated Python Integrated Development Environment (IDE) providing a wide range of essential tools for Python developers, tightly integrated to create a convenient environment for productive Python, web, and data science development. (Wikipedia)
Visual Studio Code	Visual Studio Code, also commonly referred to as VS Code, is a source-code editor made by Microsoft for Windows, Linux and macOS. Features include support for debugging, syntax highlighting, intelligent code completion, snippets, code refactoring, and embedded Git (Wikipedia)

A Brief Comparison of Python IDEs/ Editors



[Source](#)

Resources / References

- Starting out with Python (Textbook)
 - Chapter 1 and Chapter 2
- <https://pynative.com> (An online Platform to learn Python)
- <https://www.dataquest.io/blog/jupyter-notebook-tutorial/> (A Tutorial on How to use Jupyter Notebook?)